



JOKU OTSIKKO

Omar Mayani

Pro gradu -tutkielma
Kesäkuu 2020

MATEMATIIKAN JA TILASTOTIETEEN LAITOS

Tarkastajat:

Prof. Ion Petre

Markus Ylikojola

Turun yliopiston laatu järjestelmän mukaisesti tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck-järjestelmällä

TURUN YLIOPISTO, Matematiikan ja tilastotieteen laitos

Pro gradu -tutkielma

Pääaine: Matematiikka

Tekijä: Omar Mayani

Otsikko: Joku otsikko

Ohjaaja: Prof. Ion Petre

Sivumäärä: xx sivua + liitteet xx sivua

Aika: Kesäkuu 2020

Kirjoita tähän tiivistelmä. Laita `\noindent`-komento kappaleen alkuun, niin $\text{\LaTeX}$ ei sisennä ensimmäistä riviä.

Komento `\vspace` taas jättää sopivan välin kappaleiden väliin - 4mm näyttää aika hyvältä.

Kirjoita tiivistelmä napakasti ja kaikenlaista toistoa välttäen.

Asiasanat: tiivistelmäsiivu, Pro gradu -tutkielma, $\text{\LaTeX}$ -ladontajärjestelmä.



Teknologiateollisuus

Tämän pro gradu -tutkielman tutkimustyötä on tuettu Teknologiateollisuuden 100-vuotissäätiön myöntämällä apurahalla.

ON NOTATION AND TERMINOLOGY

Unless otherwise specified, we use the following notations and terminologies throughout the thesis.

$\mathbb{R}$	the set of real numbers,
$\mathcal{L}$	a loss function,
$\mathbf{T}$	the transpose of a matrix,
w	a member of a vocabulary (usually a word),
$V = \{w_1, w_2, \dots, w_{ V }\}$	a (finite) vocabulary,
$\mathbf{w}$	the vector representation of w ,

Sisällys

1	Johdanto	4
2	Diskreetit tekstiesitykset ja klassinen kielimallinnus	7
2.1	Tokenien one-hot-enkoodaus	7
2.2	N-gram-mallit	8
3	Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus	10
3.1	Upotusvektorit	10
3.2	Upotusvektorien oppiminen Word2Vec-menetelmällä	11
3.3	N-gram-malli optimointiongelmana	12
3.4	Monikerroksinen perseptroni parametrisoituna n-gram-mallina	13
4	Metodologia	16
4.1	Tutkimusasetelma	16
4.2	Tutkimusaineisto	16
4.3	Cortex Analyst ja semanttinen näkymä	16
4.4	Arviointiperusteet	18
5	Kokeet	20
6	Tulokset	23
7	Rajoitukset	28
8	Johtopäätökset	29

1 Johdanto

This section includes general description of the field and the topic of the MSc thesis.

Note that if you have used AI in your writing, you must mention it. Check for guidelines <https://utuguides.fi/artificialintelligence>. Good place to mention that AI tools have been used is here, for example, in the following form:

The ChatGPT AI has been used to enhance the language of the thesis.

Kielimalli on autoregressiivinen malli, joka ennustaa seuraavan sanan (tai tokenin) aiemman kontekstin perusteella. Kun mallin tuottama sana liitetään aiempaan kontekstiin, voidaan kutsua mallia päivitetyllä kontekstilla ja näin malli voi tuottaa pitkiäkin tekstijaksoja. Suuret kielimallit ovat viime vuosina kehittyneet pelkistä tekstin tuottajista järjestelmiksi, jotka kykenevät myös hyödyntämään ulkoisia työkaluja. Jos kielimallia suoritettava ympäristö tulkitsee tiettyjä mallin tuottamia merkinäköjä työkalukutsuina, kielimalli voi käyttää esimerkiksi laskentaa, verkkohakua tai tietokantakyselyitä.

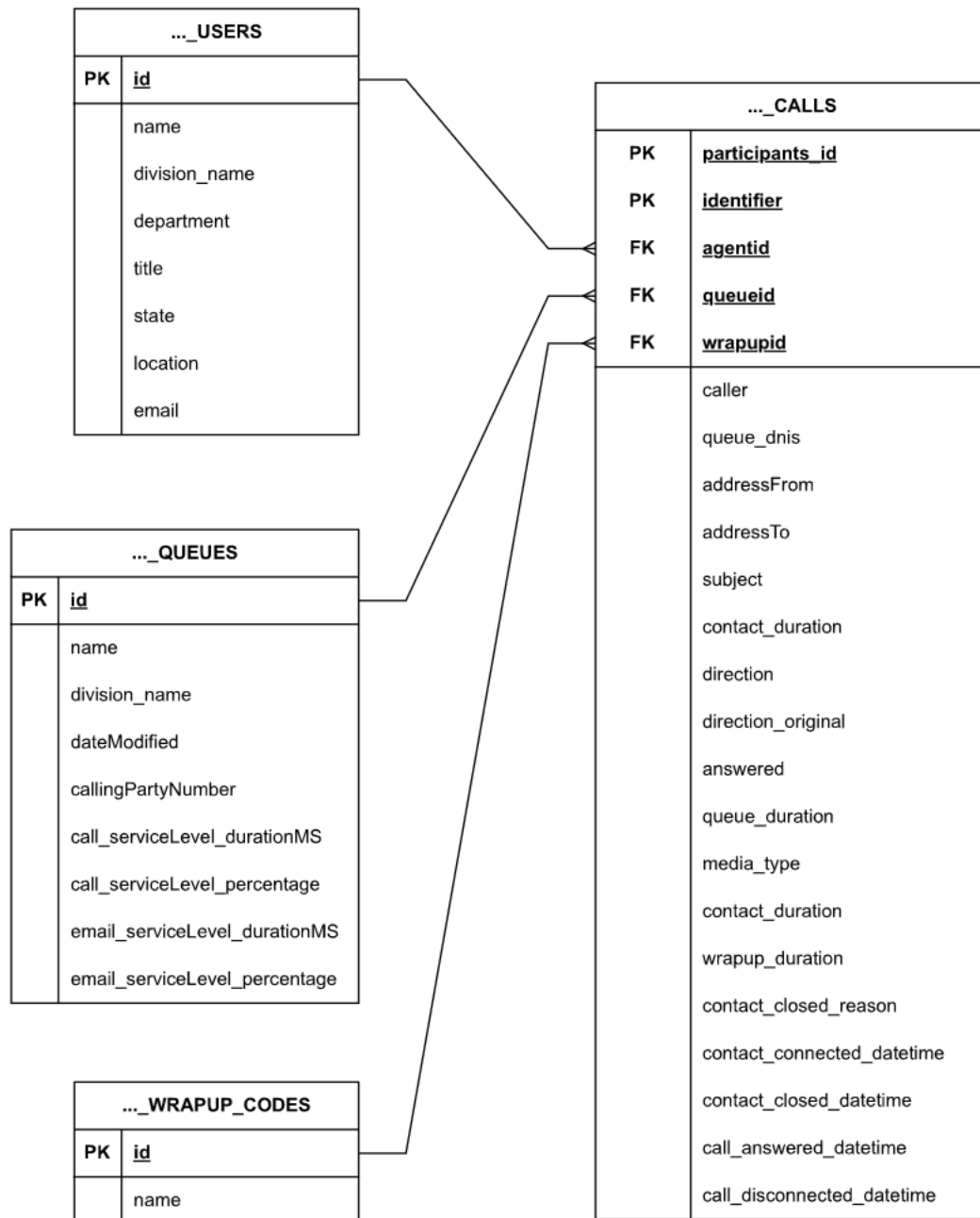
Luonnollisen kielen käyttö tietokantojen rajapintana on kiinnostava sovelluskohde, koska se voi madaltaa datan hyödyntämisen kynnyksen. Tällöin käyttäjän ei tarvitse osata SQL-kieltä eikä tuntea tietokannan rakennetta yksityiskohtaisesti, vaan hän voi esittää tiedontarpeensa tavallisella kielellä. Käytännön hyöty riippuu kuitenkin siitä, kuinka luotettavasti järjestelmä kykenee tulkitsemaan käyttäjän taroituksen ja muuntamaan sen oikeaksi SQL-kyselyksi.

Tässä tutkielmassa tarkastellaan Snowflaken Cortex Analyst -järjestelmää tapauksena. Cortex Analyst on kielimallipohjainen työkalu, joka muodostaa käyttäjän luonnollisen kielen kysymyksistä SQL-kyselyitä ja voi suorittaa niitä tietokantaa vasten. Tutkimuskohteena on puhelinkeskusdataa sisältävä tietokanta, jossa analyysin keskiössä on yksi keskusteludataa sisältävä päätason taulu sekä kolme sitä täydentävää taulua (ks. kuva 1). Järjestelmän toiminta perustuu semanttiseen näkymään, jossa taulut, kentät ja niiden merkitykset kuvataan YAML-muodossa (ks. kuva 2). Tämä näkymä toimii linkkinä käyttäjän luonnollisen kielen kysymyksen ja tietokannan rakenteen välillä.

Tutkielman tavoitteena on arvioida, kuinka hyvin Cortex Analyst soveltuu luonnollisen kielen tietokantakyselyihin ja millaisia rajoituksia sen käyttöön liittyy. Koska nykyaikaisten kaupallisten kielimallien tarkat toteutustiedot eivät yleensä ole julkisia, järjestelmän arviointi perustuu empiiriseen tarkasteluun. Tässä työssä vertaillaan nimenomaan Cortex Analystin tuottamia SQL-kyselyitä eikä niiden palauttamia vastauksia. Rajaus on perusteltu sekä aineiston luottamuksellisuuden vuoksi että siksi, että virheellisen kyselyn vaikutus lopulliseen vastaukseen riippuu myös tietokannan sisällöstä.

Tutkimusta ohjaavat seuraavat tutkimuskysymykset: Kuinka konsistentisti Cortex Analyst tuottaa SQL-kyselyitä, kun sama kysymys esitetään useita kertoja? Kuinka herkkä Cortex Analyst on sellaisille syötteiden muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä? Millaisia virhe- ja vaihtelutyyppisiä järjestelmän tuottamissa SQL-kyselyissä esiintyy?

Tutkielma etenee siten, että ensin kuvataan tutkimusasetelma, aineisto ja arviointitapa. Tämän jälkeen esitellään kokeet, joissa tarkastellaan järjestelmän konsistenssia ja herkkyyttä merkitykseltään vähäisille muutoksille. Lopuksi esitetään



Kuva 1: Tutkimuksessa käytettävän tietokannan rakenne. Taulujen nimet ollaan lyhennetty luettavuuden vuoksi.

Attribute	Value
PARTICIPANTS_ID	ea0a918f
IDENTIFIER	63c44469
QUEUEID	3b10a230
CALLERS	+123456789012
AGENTID	b7e632de
SUBJECT	Null
ADDRESS_TO	Null
ADDRESS_FROM	Null
CONTACT_DURATION	363
DIRECTION	inbound
DIRECTION_ORIGINAL	inbound
ANSWERED	1
QUEUE_DURATION	2
CALL_DURATION	316
WRAPUPID	dd91b10d
WRAPUP_DURATION	47
CONTACT_CLOSED_REASON	client
CONTACT_CONNECTED_DATETIME	2025-08-23 13:41:13.000
CONTACT_CLOSED_DATETIME	2025-08-23 13:47:28.000
CALL_ANSWERED_DATETIME	2025-08-23 13:41:25.000
CALL_DISCONNECTED_DATETIME	2025-08-23 13:46:41.000
MEDIA_TYPE	call
CONTACT_CONNECTED_DATE	2025-08-23 00:00:00.000

Taulukko 1: Esimerkki yhdestä tietokannan rivistä. Osa kentistä on anonymisoitu tietosuoja- ja luottamuksellisuussyistä.

tulokset, johtopäätökset ja tutkimuksen keskeiset rajoitukset.

2 Diskreetit tekstiesitykset ja klassinen kielimallinnus

Kielimallien yhteydessä tekstiä käsitellään tavallisesti diskreetteinä tokeneina, jotka muunnetaan laskennalliseen muotoon mallintamista varten. Olkoon $V = \{v_1, v_2, \dots, v_{|V|}\}$ sanasto eli joukko kaikista mallin käyttämistä tokeneista ja olkoon $W = (w_1, w_2, \dots, w_{|W|})$ korpus eli havaittu tokenijono. Tällöin $w_t \in V$ merkitsee korpuksen ajanhetken t tokenia. Tokenilla voidaan tarkoittaa esimerkiksi sanaa, osasanaa, merkkiä tai muuta diskreettiä tekstiyksikköä riippuen käytetystä tokenisointimenetelmästä.

2.1 Tokenien one-hot-enkoodaus

Diskreetit tokenit esitetään numeerisessa muodossa, jotta niitä voidaan hyödyntää tilastollisissa malleissa ja neuroverkoissa. Yksinkertaisin ja yleisesti käytetty esitystapa on one-hot-enkoodaus. Siinä kukin token $v_j \in V$ esitetään vektorina $x(v_j) \in \{0, 1\}^{|V|}$, jossa

$$x(v_j)_k = \begin{cases} 1, & \text{jos } k = j, \\ 0, & \text{muulloin.} \end{cases}$$

Tällainen esitys ilmaisee ainoastaan tokenin identiteetin: eri tokeneilla on toisistaan poikkeavat vektoriesitykset, mutta niihin ei sisälly eksplisiittistä semanttista tai syntaktista rakennetta. Menetelmän etuna on yksikäsitteisyys ja sen yksinkertainen yhteys sanaston indeksointiin.

Käytännön kielimallinnuksessa sanastoa täydennetään usein erikoistokeneilla, joilla kuvataan sekvenssin rakennetta ja käsitellään harvinaisia havaintoja. Tällaisia ovat esimerkiksi aloitus- ja lopetusmerkit $\langle s \rangle$ ja $\langle /s \rangle$ sekä tuntemattoman tokenin merkki $\langle unk \rangle$. Laajennettu sanasto voidaan kirjoittaa muodossa

$$V' = V \cup \{\langle s \rangle, \langle /s \rangle, \langle unk \rangle\}.$$

Aloitus- ja lopetusmerkit ovat erityisen hyödyllisiä silloin, kun mallin ennustettavaa kontekstia halutaan määritellä myös sekvenssin reuna-alueilla. Esimerkiksi n -gram-malleissa (ks. osio 2.2) kontekstin pituus on vakio, minkä vuoksi sekvenssin alku- ja loppupäähän lisätään tyypillisesti riittävä määrä erikoistokeneita, jotta kontekstit voidaan muodostaa yhtenäisesti kaikille ajanhetkille.

Koko korpus voidaan esittää one-hot-vektoreiden jonona

$$X = (x_1, x_2, \dots, x_{|W|}),$$

missä $x_t = x(w_t)$ on tokenin w_t one-hot-esitys. Jos syöte koostuu useasta tokenista, voidaan niiden one-hot-esitykset yhdistää esimerkiksi ketjuttamalla ne yhdeksi syötevektoriksi

$$\tilde{x}_t = [x_{t-n+1}; x_{t-n+2}; \dots; x_{t-1}] \in \{0, 1\}^{(n-1)|V|}.$$

Tämä muodostaa luonnollisen syötteen monille kontekstipohjaisille malleille, sillä se säilyttää kontekstin tokenijärjestyksen eksplisiittisesti.

One-hot-enkoodaus muodostaa siten perustavanlaatuisen sillan diskreetin tekstin ja numeeristen kielimallien välille. Se tarjoaa täsmällisen ja muodollisesti yksinkertaisen esitystavan, jonka varaan voidaan rakentaa sekä tilastollisia n -gram-malleja että myöhempiä neuroverkkopohjaisia kielimalleja.

2.2 N-gram-mallit

N-gram-malli on klassinen tilastollinen kielimalli, jossa seuraavan sanan tai symbolin ennustaminen perustuu sitä edeltävään rajattuun kontekstiin. Mallin taustalla on $(n - 1)$:n kertaluvun Markovin oletus, jonka mukaan nykyinen tokeni riippuu vain edeltävistä $(n - 1)$ tokenista eikä koko aiemmasta historiallisesta kontekstistä [8, 7]. Tällöin voidaan approksimoida ehdollinen todennäköisyys

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}),$$

missä w_t on ajanhetken t tokeni ja $(w_{t-n+1}, \dots, w_{t-1})$ on sitä edeltävät $(n - 1)$ tokenia.

Olkoon $V = \{v_1, v_2, \dots, v_{|V|}\}$ sanasto, jossa $|V|$ on sanaston koko, ja $W = (w_1, w_2, \dots, w_{|W|})$ havaintoaineisto. N-gram-malli voidaan toteuttaa muodostamalla lukumatriisi $C \in \mathbb{R}^{|V|^{n-1} \times |V|}$, jossa $C_{c,j}$ edustaa sitä, kuinka monta kertaa kontekstia c seuraa tokeni v_j havaintoaineistossa. Formaalisti

$$C_{c,j} = |\{t \in \mathbb{N} \mid (w_{t-n+1}, \dots, w_{t-1}) = c \text{ ja } w_t = v_j\}|.$$

Kun lukumatriisin C rivit normalisoidaan jakamalla jokainen alkio rivinsä summalla, saadaan frekvenssimatriisi $F \in \mathbb{R}^{|V|^{n-1} \times |V|}$, jossa

$$F_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|V|} C_{c,k}},$$

ja joka edustaa ehdollista todennäköisyyttä

$$P(w_t = v_j \mid (w_{t-n+1}, \dots, w_{t-1}) = c).$$

On helppo osoittaa, että nämä ehdolliset todennäköisyydet vastaavat suurimman uskottavuuden estimaatteja [5]. N-gram-mallin oletuksen mukaisesti koko havaintoaineiston todennäköisyys voidaan kirjoittaa ehdollisten todennäköisyyksien tulona

$$P(W) \approx \prod_{t=1}^{|W|} P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Kiinteän kontekstin c tapauksessa havaintojen lukumäärät

$$(C_{c,1}, C_{c,2}, \dots, C_{c,|V|})$$

muodostavat multinomijakauman otoksen, jossa parametreina ovat todennäköisyydet

$$\theta_c = (\theta_{c,1}, \theta_{c,2}, \dots, \theta_{c,|V|}), \quad \sum_{j=1}^{|V|} \theta_{c,j} = 1.$$

Tällöin kyseisen kontekstin osalta uskottavuusfunktio voidaan kirjoittaa muodossa

$$L(\theta_c) = \prod_{j=1}^{|V|} \theta_{c,j}^{C_{c,j}}.$$

Tämän funktion maksimoiminen on ekvivalenttia sen logaritmin maksimoimiselle, joten saadaan Lagrangen funktio

$$\mathcal{L}_{\text{Lag}}(\theta_c, \lambda) = \sum_{j=1}^{|V|} C_{c,j} \log \theta_{c,j} + \lambda \left(1 - \sum_{j=1}^{|V|} \theta_{c,j} \right).$$

Derivoimalla saadaan

$$\frac{\partial \mathcal{L}_{\text{Lag}}}{\partial \theta_{c,j}} = \frac{C_{c,j}}{\theta_{c,j}} - \lambda = 0, \quad j = 1, 2, \dots, |V|,$$

josta seuraa

$$\theta_{c,j} = \frac{C_{c,j}}{\lambda}.$$

Kun tämä sijoitetaan normalisaatioehtoon, saadaan

$$\lambda = \sum_{k=1}^{|V|} C_{c,k},$$

ja siten

$$\theta_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|V|} C_{c,k}}, \quad j = 1, 2, \dots, |V|.$$

Tämä on täsmälleen rivikohtaisesti normalisoitu frekvenssimatriisi. Näin ollen n -gram-mallin suurimman uskottavuuden estimaatti ja laskumatriisiin perustuva estimaatti ovat yhtäpitäviä.

Tekstin generointi n -gram-mallilla tapahtuu iteratiivisesti hyödyntämällä frekvenssimatriisia F . Ensin valitaan alkukonteksti, esimerkiksi erikoismerkeillä aloitettu $(n-1)$ -tokenin pituinen alkujakso. Tämän jälkeen mallin tämänhetkistä kontekstia vastaava rivi F_c tulkitaan todennäköisyysjakaumaksi, josta seuraava tokeni valitaan satunnaisotannalla. Kun tokeni w_t on generoitu, konteksti päivitetään siirtämällä ikkuna yhden askeleen eteenpäin siten, että uusi konteksti muodostuu edeltävistä $(n-1)$ generoidusta tokenista. Prosessia jatketaan, kunnes generoidaan päätemerkki tai saavutetaan haluttu tekstin pituus.

Vaikka lähestymistapa on käsitteellisesti yksinkertainen ja tulkittava, siihen liittyy merkittäviä laskennallisia haasteita erityisesti kontekstikoon n kasvaessa. Tämän seurauksena lukumatriisista $C \in \mathbb{R}^{|V|^{n-1} \times |V|}$ tulee nopeasti erittäin suuri ja käytännöllisissä sovelluksissa hyvin harva, koska vain pieni osa mahdollisista konteksteista esiintyy tyypillisessä havaintoaineistossa. Tämä lisää sekä muistintä laskentatehon tarvetta [7].

Diskreetit esitykset ja frekvenssipohjaiset n -gram-mallit tarjoavat käsitteellisesti selkeän lähtökohdan kielimallinnukselle. Niiden keskeinen rajoite on kuitenkin heikko yleistyskyky harvinaisiin ja havaitsemattomiin konteksteihin. Tämän vuoksi on hyödyllistä tarkastella samaa ongelmaa optimointitehtävänä, mikä muodostaa luontevan siirtymän kohti jatkuvia vektoriesityksiä ja parametrisoituja neuroverkkopohjaisia malleja.

3 Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus

3.1 Upotusvektorit

One-hot-esitys tarjoaa yksikäsitteisen tavan kuvata tokeneita, mutta sen heikkoutena on, ettei se sisällä eksplisiittistä tietoa tokenien välisistä suhteista. Jokaisen tokenin vektorikuvaus on ortogonaalinen kaikkiin muihin nähden, jolloin esimerkiksi sanan *kissa* suhde sanaan *koira* on rakenteellisesti sama kuin sen suhde sanaan *tietokone*. Tämän rajoitteen ylittämiseksi käytetään upotusvektoreita (engl. *embedding vectors*), joissa kukin token kuvataan tiheällä reaaliarvoisella vektorilla.

Olkoon edelleen $V = \{v_1, v_2, \dots, v_{|V|}\}$ sanasto. Tällöin tokenille v_j määritellään upotus

$$e(v_j) \in \mathbb{R}^d,$$

missä $d \ll |V|$ on upotustilan dimensio.

Upotusvektorit voidaan esittää myös matriisimuodossa. Olkoon

$$E \in \mathbb{R}^{d \times |V|}$$

upotusmatriisi, jonka sarakkeet vastaavat sanaston tokeneita. Tällöin tokenin v_j upotus saadaan one-hot-vektorin $x(v_j)$ avulla muodossa

$$e(v_j) = Ex(v_j).$$

Koska one-hot-vektori sisältää täsmälleen yhden epä-nollan komponentin, tämä kertolasku palauttaa matriisista E juuri tokenia v_j vastaavan sarakkeen.

Upotusvektorien keskeinen etu liittyy siihen, että samankaltaisten tokenien upotukset voivat sijaita lähellä toisiaan upotustilassa. Tällöin tokenien geometrinen läheisyys voi heijastaa niiden semanttista tai syntaktista samankaltaisuutta. Esimerkiksi sanat “kuningas” ja “kuningatar” voivat saada upotukset, jotka ovat lähellä toisiaan, koska ne esiintyvät samankaltaisissa konteksteissa ja jakavat merkityksellisiä piirteitä. Upotusvektorit tarjoavat siten rikkaamman ja joustavamman esityksen tokenien välisistä suhteista kuin one-hot-esitys, jossa kaikki tokenit ovat yhtä kaukana toisistaan.

Käytännön kielimallinnuksessa upotukset opitaan tavallisesti tehtäväkohtaisesti osana laajempaa optimointia. Jos korpus on $W = (w_1, w_2, \dots, w_{|W|})$, voidaan jokainen tokeni $w_t \in V$ korvata sitä vastaavalla upotusvektorilla $e(w_t)$. Tällöin koko korpusta voidaan tarkastella tiheiden vektorien jonona

$$E_W = (e(w_1), e(w_2), \dots, e(w_{|W|})).$$

Tämä esitystapa on erityisen hyödyllinen neuroverkoissa, joissa upotukset muodostavat syötteen myöhemmille kerroksille ja joissa opittavat parametrit voivat hyödyntää tokenien välisiä yhteyksiä yhteisen esitystilan kautta.

Upotusvektorit ovat myös yhteensopivia erikoistokenien kanssa. Esimerkiksi aloitusmerkki $\langle s \rangle$, lopetusmerkki $\langle /s \rangle$ ja tuntematon token $\langle unk \rangle$ voidaan sisällyttää laajennettuun sanastoon

$$V' = V \cup \{\langle s \rangle, \langle /s \rangle, \langle unk \rangle\},$$

jolloin niille voidaan oppia omat upotuksensa samalla tavalla kuin muillekin tokenille. Näin myös sekvenssien rakenteelliset elementit voidaan käsitellä yhtenäisesti muun sanaston kanssa.

Upotusvektorit muodostavat siten luonnollisen jatkeen one-hot-esitykselle. Siinä missä one-hot kuvaa tokenin pelkkänä identiteettinä, upotusvektori tarjoaa tiiviin ja optimoitavan esityksen, joka soveltuu tehokkaasti neuroverkkopohjaisiin kielimalleihin.

3.2 Upotusvektorien oppiminen Word2Vec-menetelmällä

Edellä esitetty upotusvektorit eivät ole ennalta annettuja, vaan ne opitaan tyypillisesti korpuksesta optimointitehtävän avulla. Yksi tunnetuimmista ja vaikuttavimmista menetelmistä tähän tarkoitukseen on Word2Vec, jonka perusarkkitehtuurit esittelevät Mikolov ym. työssään *Efficient Estimation of Word Representations in Vector Space* [9]. Menetelmän keskeinen ajatus on oppia sellaiset vektoriesitykset, että samankaltaisissa konteksteissa esiintyvät sanat saavat samankaltaiset upotukset. Tämä perustuu distributionaaliseen periaatteeseen, jonka mukaan sanan merkitystä voidaan approksimoida sen käyttöympäristöjen perusteella.

Olkoon korpus edelleen $W = (w_1, w_2, \dots, w_{|W|})$, missä $w_t \in V$. Word2Vec-malleissa tarkastellaan tavallisesti kunkin sanan ympärille muodostettua kontekstia, jonka laajuus määrätään ikkunaparametrilla $c \in \mathbb{N}$. Tällöin ajanhetken t kontekstiksi saadaan esimerkiksi joukko

$$C_t = \{w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}\},$$

olettaen, että indeksit pysyvät korpuksen rajojen sisällä. Word2Vec-menetelmän kaksi keskeistä arkkitehtuuria ovat jatkuvien sanojen pussimalli (*continuous bag-of-words*, CBOW) ja Skip-gram [9].

CBOW-mallissa ennustetaan keskimäinen sana sen kontekstin perusteella. Toisin sanoen mallin tavoitteena on approksimoida todennäköisyyttä

$$p(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}).$$

Kontekstin tokenit muunnetaan ensin upotusvektoreiksi, minkä jälkeen niiden esitykset yhdistetään tavallisesti summaamalla tai keskiarvoistamalla. Saatua kontekstivektoria käytetään ennustamaan kohdesana koko sanaston yli. Mikolov ym. [9] korostavat, että CBOW on laskennallisesti tehokas erityisesti suurilla aineistoilla, koska se aggregoi kontekstin yhdeksi esitykseksi eikä käsittele jokaista kontekstisanaa täysin erillisenä ennustustehtävänä.

Skip-gram-mallissa ennustussuunta on päinvastainen: keskimäisen sanan avulla pyritään ennustamaan sitä ympäröivät kontekstisanat. Tavoitteena on siis maksimoida todennäköisyydet

$$p(w_{t+j} \mid w_t), \quad -c \leq j \leq c, j \neq 0.$$

Koko korpuksen yli tämä voidaan kirjoittaa maksimoitavana tavoitefunktiona muodossa

$$\sum_{t=1}^{|W|} \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} \mid w_t).$$

Skip-gram soveltuu hyvin tilanteisiin, joissa halutaan oppia laadukkaita esityksiä myös harvinaisemmille sanoille, sillä kukin havaittu sana toimii useiden kontekstien ennustamisen lähtökohtana [9].

Todennäköisyys $p(w_O \mid w_I)$, missä w_I on syötesana ja w_O kohdesana, voidaan määritellä softmax-funktion avulla. Jos syötesanaa w_I vastaa vektori v_{w_I} ja kohdesanaa w_O vastaava ulostulovektori on u_{w_O} , saadaan

$$p(w_O \mid w_I) = \frac{\exp(u_{w_O}^\top v_{w_I})}{\sum_{w \in V} \exp(u_w^\top v_{w_I})}.$$

Tässä nimittäjän summa ulottuu koko sanastoon, mikä tekee täsmällisestä optimoinnista laskennallisesti raskasta suurilla sanastoilla. Mikolov ym. [9] esittävät tehokkaita ratkaisuja tähän ongelmaan, jotta sanavektoreita voidaan oppia erittäin suurista korpuksista käytännöllisin resurssein.

Word2Vec-mallissa opitaan käytännössä kaksi parametrimatriisia: syötevektorit ja ulostulovektorit. Jokaiselle sanaston sanalle $v_j \in V$ voidaan siis assosoida ainakin yksi tiheä vektorikuvaus. Oppimisen jälkeen näistä käytetään tavallisesti sanan syötevektoria tai vaihtoehtoisesti syöte- ja ulostulovektoreiden yhdistelmää sanan lopullisena esityksenä. Näin muodostuvat upotukset heijastavat sanojen yhteisesiintymisrakenteita korpuksessa: vektorit, jotka esiintyvät samankaltaisissa käyttöympäristöissä, sijoittuvat tyypillisesti lähelle toisiaan upotustilassa.

Mikolov ym. [9] osoittavat lisäksi, että tällä tavoin opitut vektoriesitykset säilyttävät kielellisesti mielekkäitä säännönmukaisuuksia. Sanavektorien väliset erot voivat vastata semanttisia tai syntaktisia suhteita, mikä näkyy esimerkiksi analogiatehtävissä. Tämän vuoksi Word2Vec ei ole ainoastaan tehokas optimointimenetelmä, vaan myös empiirisesti toimiva tapa rakentaa sellaisia upotuksia, jotka tiivistävät sanojen välisiä suhteita hyödyllisellä tavalla.

Vaikka Word2Vec kehitettiin alun perin sanojen vektoriesitysten oppimiseen, sen perusajatus yleistyy myös laajemmin tokenipohjaiseen mallinnukseen. Menetelmä havainnollistaa, kuinka diskreetistä tokenijonosta voidaan oppia jatkuvaesityksiä pelkästään kontekstuaalisen ennustetehtävän avulla. Tässä mielessä se muodostaa tärkeän sillan klassisten jakaumallisten sanamallien ja myöhempien neuroverkko-pohjaisten kielimallien välille.

3.3 N-gram-malli optimointiongelmana

Sen sijaan, että mallin parametrit määriteltäisiin suoraan havaintofrekvenssien avulla, n -gram-malli voidaan esittää optimointitehtävänä. Tällöin tavoitteena on löytää parametrit θ , jotka maksimoivat havaintoaineiston uskottavuuden $P(W; \theta)$.

Koska koneoppimisessa optimointiongelmat esitetään tyypillisesti minimointimuodossa, on luonnollista määritellä häviöfunktioksi negatiivinen log-uskottavuus. Kun havaintoaineisto koostuu konteksti-seuraava tokeni -pareista, voidaan kirjoittaa

$$J(\theta; W) = -\frac{1}{|W|} \sum_{t=1}^{|W|} \log P_\theta(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Tämä suure kuvaa keskimääräistä negatiivista log-todennäköisyyttä havaintoaineistossa, ja sen minimointi on täsmälleen ekvivalenttia uskottavuuden maksimoinnin kanssa. Toisin sanoen

$$\arg \max_{\theta} P(W; \theta) = \arg \min_{\theta} J(\theta; W).$$

Mikäli n -gram-malli parametrisoidaan kontekstikohtaisilla todennäköisyyksillä $\theta_c = (\theta_{c,1}, \dots, \theta_{c,|V|})$, voidaan koko havaintoaineiston uskottavuus kirjoittaa muodossa

$$P(W; \theta) = \prod_c \prod_{j=1}^{|V|} \theta_{c,j}^{C_{c,j}},$$

missä $C_{c,j}$ on kontekstin c jälkeen esiintyvän tokenin v_j lukumäärä. Tällöin negatiivinen log-uskottavuus saa muodon

$$J(\theta; W) = -\frac{1}{|W|} \sum_c \sum_{j=1}^{|V|} C_{c,j} \log \theta_{c,j}.$$

Kunkin kontekstin parametrit ovat toisistaan riippumattomia, ja ne on lisäksi rajoitettu todennäköisyysjakaumiksi siten, että

$$\sum_{j=1}^{|V|} \theta_{c,j} = 1, \quad \theta_{c,j} \geq 0.$$

Näin ollen jokainen konteksti muodostaa erillisen multinomiaalisen optimointiongelman, jonka ratkaisu saadaan maksimoimalla vastaava log-uskottavuus. Tämä johtaa täsmälleen samaan estimaattiin kuin rivikohtaisesti normalisoitu frekvenssimatriisi:

$$\theta_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|V|} C_{c,k}}.$$

Tämä tulos osoittaa, että lukumatriisiin perustuva n -gram-malli on yhtäpitävä suurimman uskottavuuden estimoinnin kanssa. Samalla käy kuitenkin ilmi mallin keskeinen rajoite: jokaiselle mahdolliselle kontekstille tarvitaan oma todennäköisyysjakaumansa. Kun kontekstin pituus kasvaa, mahdollisten kontekstien määrä kasvaa nopeasti, mikä tekee mallista harvan ja vaikeasti skaalautuvan. Lisäksi harvinaisten tai kokonaan havaitsemattomien kontekstien tapauksessa suora frekvenssiperusteen estimaatti ei yleisty hyvin.

Optimointinäkökulma tekee näkyväksi, että kielimallinnuksen ytimessä on ehdollisten todennäköisyyksien parametrinen estimointi. Tästä näkökulmasta on luontevaa siirtyä malleihin, joissa todennäköisyyksiä ei enää talleteta erillisinä taulukoituina jakaumina, vaan ne opitaan jatkuvassa avaruudessa parametrisoidun funktion avulla.

3.4 Monikerroksinen perseptroni parametrisoituna n -gram-mallina

Kun kiinteäpituinen n -gram-konteksti esitetään one-hot-vektoreiden sijasta tiheinä upotusvektoreina, voidaan seuraavan tokenin ehdollista jakaumaa approksimoida

parametrisoidulla neuroverkolla. Yksi yksinkertainen ja historiallisesti tärkeä ratkaisu tähän on monikerroksinen perseptroni (engl. *multilayer perceptron*, MLP).

MLP on eteenpäinsyöttävä keinotekoinen neuroverkkomalli, joka muodostaa pohjan monille syvän oppimisen arkkitehtuureille. Matemaattisesti MLP voidaan ymmärtää funktion approksimaattorina, joka muodostaa kuvauksen syöteavaruudesta tulosteavaruuteen. Tämä kuvaus toteutetaan yhdistettynä funktiona, joka koostuu useista peräkkäisistä laskentakerroksista. Kerrokset rakentuvat affineista muunnoksista ja niitä seuraavista epälineaarista aktivaatiofunktioista, mikä mahdollistaa epälineaaristen riippuvuuksien mallintamisen. MLP:n toimintaa ohjaavat opittavat parametrit, tyypillisesti painomatriisit ja bias-vektorit, joiden arvot estimoidaan datasta minimoimalla valittua kohdefunktiota numeeristen optimointialgoritmien, kuten gradienttimenetelyn, avulla.

Kielimallinnuksen kontekstissa MLP saa syötteenään kielellisen kontekstin, eli edeltävien tokenien jonon, ja tuottaa tulosteenään ehdollisen todennäköisyysjakauman sekvenssin seuraavalle tokenille. Formaalisti kielimallinnukseen sovellettu MLP voidaan esittää funktiona

$$f_\theta : C \rightarrow [0, 1]^{|V|},$$

missä C kuvaa mahdollisten kontekstien avaruutta, V on mallin sanasto ja θ viittaa MLP:n opittaviin parametreihin. Kun mallille syötetään edeltävistä tokeneista koostuva konteksti $c = (w_{t-n+1}, \dots, w_{t-1})$, funktio tuottaa todennäköisyysjakauman

$$p_\theta(\cdot \mid c) = f_\theta(c),$$

joka kuvaa kunkin sanastoon kuuluvan tokenin esiintymistodennäköisyyttä syöte-kontekstin jatkeena.

Konteksti voidaan esittää esimerkiksi one-hot-vektoreina tai tiheinä upotusvektoreina. Tällöin MLP voi ensin muodostaa kontekstista piirre-esityksen ja sen jälkeen muuntaa sen ulostuloksi, joka normalisoidaan softmax-funktion avulla todennäköisyysjakaumaksi:

$$p_\theta(w_t = v_j \mid c) = \frac{\exp(z_j)}{\sum_{k=1}^{|V|} \exp(z_k)},$$

missä z_j on MLP:n antama ennen normalisointia oleva pistemäärä tokenille v_j .

Yksinkertaisessa MLP-pohjaisessa n -gram-mallissa konteksti muunnetaan ensin piilokerroksen esitykseksi esimerkiksi muodossa

$$h = \phi(Wx + b),$$

missä x on kontekstin syötevektori, W painomatriisi, b bias-termi ja ϕ epälineaarinen aktivaatiofunktio. Tämän jälkeen piilokerroksesta lasketaan tokenien pistemäärät ja niistä edelleen softmax-jakauma.

Mallin oppiminen perustuu samaan negatiiviseen log-uskottavuuteen kuin aiemmin:

$$J(\theta) = -\frac{1}{|W|} \sum_{t=1}^{|W|} \log p_\theta(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Erona perinteiseen frekvenssipohjaiseen n -gram-malliin, joka estimoi jokaiselle mahdolliselle diskreetille kontekstille oman erillisen parametrivektorinsa, MLP approksimoi todennäköisyyksiä jatkuvassa avaruudessa. Koska erilliset tokenit projisoidaan ensin matalaulotteisiksi tiheiksi upotusvektoreiksi $v \in \mathbb{R}^d$, malli oppii ilmaisemaan sanojen semanttisia ja syntaktisia samankaltaisuuksia vektoreiden välisinä etäisyyksinä. MLP:n approksimoima funktio on jatkuva ja tyypillisesti sileä siinä mielessä, että pienet muutokset syöteavaruudessa voivat johtaa samankaltaisiin ulostulojakaumiin. Tästä syystä malli kykenee yleistämään myös harvinaisiin tai aiemmin havaitsemattomiin konteksteihin hyödyntämällä sellaisten koulutusdatassa esiintyneiden kontekstien rakennetta, joiden esitykset sijaitsevat lähellä niitä yhteisessä avaruudessa [4]. Tämä lieventää klassisia n -gram-malleja vaivaavaa nol-latodennäköisyyksien ongelmaa ilman erillisiä tasoitusmenetelmiä.

Frekvenssipohjaista mallia koskevat tekniset rajoitteet, kuten eksponentiaalinen muistitarpeen kasvu ja lukumatriisin harvuus, voidaan siten osittain ratkaista parametrisoimalla n -gram-malli MLP:n avulla. MLP-rakenteessa opittavat painomatriisit ja bias-vektorit jakautuvat kaikkien kontekstien kesken. Siten jokainen harjoitusnäyte päivittää koko neuroverkon parametreja, mikä parantaa myös sellaisten lauseiden ennustamista, joita ei ole esiintynyt opetusdatassa, mutta jotka jakavat samankaltaisia piirteitä opittujen esitysten kautta. Tämä tekee MLP-pohjaisesta n -gram-mallista huomattavasti joustavamman, muistikapasiteetiltaan tehokkaamman ja yleistyskykyisemmän kuin perinteinen frekvenssiperusteinen taulukkolukuesitys [6, 7].

4 Metodologia

4.1 Tutkimusasetelma

Tutkimus toteutetaan empiirisenä tapaustutkimuksena, jossa arvioidaan yhden kielimallipohjaisen järjestelmän toimintaa rajatussa sovelluskontekstissa. Tavoitteena ei ole rakentaa uutta mallia tai vertailla useita eri järjestelmiä keskenään, vaan tarkastella, miten Cortex Analyst suoriutuu käytännöllisestä tehtävästä, jossa luonnollisen kielen kysymykset muunnetaan SQL-kyselyiksi. Tapaustutkimus soveltuu tähän tarkoitukseen hyvin, koska se mahdollistaa järjestelmän toiminnan yksityiskohtaisen tarkastelun todellisessa kaupallisessa ympäristössä.

Tutkimuksen analyysiyksikkönä on yksittäinen käyttäjän luonnollisen kielen kysymys ja sitä vastaava Cortex Analystin tuottama SQL-kysely. Arviointi kohdistuu siihen, vastaako muodostettu kysely käyttäjän tarkoitusta, hyödyntääkö se oikeita tauluja ja kenttiä sekä noudattaako se tietokannan rakennetta. Näin tutkimus keskittyy suoraan siihen vaiheeseen, jossa kielimallin tulkinta muuttuu koneellisesti suoritettavaksi tietokantakyselyksi.

4.2 Tutkimusaineisto

Tutkimuksen aineistona toimii puhelinkeskusdataa sisältävä tietokanta, joka noudattaa kuvassa 1 esitettyä rakennetta. Tarkastelun keskiössä on yksi keskusteludataa sisältävä päätason taulu, jota täydentävät kolme liitännäistaulua. Liitännäistaulujen avulla voidaan tarkentaa päätason taulun yksittäisten kenttien sisältöä ja merkitystä. Tietokannan rakenne on siten riittävän monipuolinen, jotta voidaan tarkastella sekä yksinkertaisia että vaativampia kyselytilanteita, kuten taululiitoksia, suodatuksia ja aggregaatioita.

Aineisto sisältää kaupallisesti arkaluonteista tietoa, minkä vuoksi varsinaisia kyselytuloksia ei julkaista. Tästä syystä tutkimuksessa vertaillaan järjestelmän tuottamia SQL-kyselyitä eikä niiden palauttamia tietosisältöjä. Ratkaisu tukee myös analyysin johdonmukaisuutta, koska virheellisen kyselyn vaikutus lopputulokseen ei ole aina pääteltävissä pelkästään syntaksista, vaan riippuu osittain tietokannan sisällöstä.

4.3 Cortex Analyst ja semanttinen näkymä

Tutkimuksessa käytettävä järjestelmä on Snowflaken Cortex Analyst. Järjestelmän tarkoituksena on mahdollistaa tietokantatiedon hyödyntäminen luonnollisen kielen kautta siten, että käyttäjän ei tarvitse tuntea SQL-kieltä tai tietokannan skeemaa yksityiskohtaisesti. Cortex Analyst muodostaa käyttäjän kysymyksestä SQL-kyselyn ja voi suorittaa sen tietokantaa vasten.

Cortex Analystin toiminta perustuu semanttiseen näkymään, jossa tietokannan taulut, kentät ja niiden merkitykset kuvataan YAML-muotoisena määrittelynä (ks. kuva 2). Semanttinen näkymä tarjoaa kielimallille rakenteellisen kuvauksen siitä, mitä tietoa tietokannassa on saatavilla ja miten eri taulut liittyvät toisiinsa. Näin järjestelmä pystyy muodostamaan syntaktisesti valideja SQL-kyselyitä ja kohdistamaan kyselyn oikeisiin tietokannan osiin. Semanttisen näkymän laatu on keskeinen

osa järjestelmän toimivuutta, koska kielimallin tulkinta perustuu olennaisesti siihen, miten skeema on kuvattu.

Tutkimuksessa käytetty semanttinen näkymä on tutkijan rakentama yhteistyössä yksityisen IT-konsulttiyrityksen Knowitin ja yhden heidän asiakkaan kanssa osana hanketta, jossa pyritään hyödyntämään kielimalleja helpottamaan tietokannan käyttöä.

Tässä tutkimuksessa käytetty semanttinen näkymä kuvailee kaikki kuvassa 1 esiintyvät taulut, niiden kentät ja suhteet toisiinsa sekä useita kentistä laskettuja metriikoita. Kuvassa 2 on esitetty yksinkertaistettu versio käytetystä semanttisesta näkymästä.

```
name: SEM_EXAMPLE
description: |-
  Semantic model that models conversations data from Genesys
  Cloud contact center software.
tables:
  - name: CONV
    description: |-
      All Genesys conversations.
    base_table:
      database: DEV
      schema: DBT_OM
      table: FACT_GENESYS__CONVERSATIONS_STANDARD
    dimensions:
      - name: C_ADDRESS_FROM
        description: Source address for the interaction. Only
          present in email contacts..
        expr: conv.address_from
        data_type: VARCHAR(16777216)
    facts:
      - name: CALL_DURATION
        description: Duration of the actual call in seconds.
          The time the customer speaks to an agent
        expr: conv.call_duration
        data_type: NUMBER(38,0)
        access_modifier: public_access
    metrics:
      - name: AVERAGE_CALL_DURATION
        description: Average call duration in seconds
        expr: sum(conv.call_duration)
        access_modifier: public_access
    primary_key:
      columns:
        - PARTICIPANTS_ID
        - IDENTIFIER
```

Kuva 2: Esimerkki semanttisesta näkymästä. Esimerkki on yksinkertaistettu versio tutkimuksessa käytetystä näkymästä.

Kokeiden aikana Cortex Analyst käytti taustallaan Anthropicin Claude Sonnet 4 -mallia. Tutkimuksen tulokset kuvaavat ensisijaisesti juuri tämän järjestelmän ja mallin yhdistelmän toimintaa. Huhtikuussa 2026 Cortex Analyst voi käyttää uudempaa Claude Sonnet 4.6 -mallia [3]. Tuloksia ei siksi voida sellaisenaan yleistää kaikkiin kielimalleihin tai muihin luonnollisen kielen tietokantarajapintoihin.

4.4 Arviointiperusteet

Tuotettuja SQL-kyselyitä arvioidaan useiden toisiaan täydentävien kriteerien avulla. Ensinnäkin tarkastellaan, valitseeko järjestelmä oikeat taulut ja käyttääkö se oikeita liitoksia silloin, kun käyttäjän kysymys edellyttää useamman taulun yhdistämistä. Toiseksi arvioidaan suodatusehtoja, erityisesti where-lauseiden sisältöä, koska pienikin virhe niissä voi muuttaa kyselyn merkitystä olennaisesti. Lisäksi huomioidaan aggregaatiot, duplikaattien käsittely ja sarakevalinnat.

Syntaktista validiutta ei tarkastella erikseen, koska kaikki järjestelmän tuottamat kyselyt olivat syntaktisesti valideja.

Arvioinnissa erotetaan toisistaan neljä poikkeamatyyppiä. Kysely luokitellaan korrektiksi, jos sen perusteella saatava vastaus vastaisi esitettyyn luonnollisen kielen kysymykseen oikein ja huomioisi ne implisiittiset taustaoletukset, jotka ovat sovellusalueen näkökulmasta perusteltuja.

Tutkittava tietokanta sisältää puhelinkeskusdataa, joten erityisesti puhelun suuntaan liittyvät taustaoletukset ovat keskeisiä. Esimerkiksi palvelutasoa koskevan kysymyksen implisiittisenä taustaoletuksena pidetään sitä, että tarkastelu kohdistuu vain saapuviin kontakteihin. Korrekti kysely huomioi tällaiset taustaoletukset ja vastaa käyttäjän tiedontarpeeseen oikein.

Kysely merkitään melkein korrektiksi, jos generoitu SQL vastaa olennaisilta osin kysymykseen, mutta sisältää teknisen puutteen. Esimerkiksi kysely, josta puuttuu NULLIF -lause jakolaskun nimittäjästä, vastaa sisällöllisesti samaan kysymykseen kuin NULLIF-avainsanan sisältävä kysely, mutta voi erikoistapauksessa johtaa virheeseen. Tällainen kysely luokitellaan melkein korrektiksi.

Kysely luokitellaan virheelliseksi, jos se ei vastaa kysymykseen merkityksellisellä tavalla. Esimerkiksi kysely, joka käyttää väärää suodatusta tai josta puuttuu olennainen taulu tai kenttä, merkitään virheelliseksi.

Jos malli ei tuota lainkaan SQL-kyselyä, vaan ilmoittaa, että kysymykseen ei voida vastata, tämä luokitellaan erikseen ei kyselyä -kategoriaksi. Kaikkiin tutkimuksessa käytettyihin kysymyksiin on mahdollista vastata käytettävissä olevan aineiston ja semanttisen näkymän perusteella, joten tähän kategoriaan kuuluvat vastaukset ovat virheellisiä. Ne on kuitenkin mielekästä erottaa muista virheellisistä vastauksista, koska ne eivät tuota harhaanjohtavia tuloksia.

Samaan luonnollisen kielen kysymykseen voi olla useita oikeellisia SQL-vastauksia. Esimerkiksi kysymykseen How many contacts did we handle in January 2026? järjestelmä voi tuottaa kyselyn, joka palauttaa yhden kokonaisluvun, tai kyselyn, joka palauttaa taulun sisältäen kontaktien määrän tammi-, helmi- ja maaliskuulle, joista käyttäjä voi poimia vastauksen. Tässä tutkielmassa kysely tulkitaan oikeelliseksi siinä tapauksessa, että sen tuottama vastaus sisältää käyttäjän tiedontarpeen.

Tästä johtuen oikeellisuuden lisäksi tutkimuksen keskeinen kiinnostuksen kohde

on kielimallin tuottamien kyselyiden vaihtelu. Tämän vuoksi analysoidaan myös, kuinka monta uniikkia SQL-kyselyä järjestelmä tuottaa kutakin kysymystä kohden, kun sama luonnollisen kielen kysymys tai jokin sen variaatioista esitetään useita kertoja.

Kyselyä ei pidetä uniikkina, jos semanttisesti ekvivalentti kysely on tuotettu aiemmin, paitsi siinä tapauksessa, että toinen kysely hyödyntää semanttisessa näkymässä valmiiksi määritettyä metriikkaa ja toinen tuottaa laskukaavan, joka on ekvivalentti semanttisessa näkymässä määritellyn metriikan kanssa. Esimerkiksi jos semanttiseen näkymään ollaan määritellyt metriikka `total_calls` kaavalla `select sum(*) from conv`, niin kyselyt `select sum(*) from conv` ja `select total_calls from semantic_view` nähdään toisistaan uniikkeina.

Päätös perustellaan sillä, että nämä kaksi lähestymistapaa ovat olennaisesti erilaiset (käytettävissä olevan kontekstin soveltaminen vs. päättely). Vaikka kyselyt ovat sisällöllisesti semanttisesti samankaltaisia, niiden muodostaminen edellyttää erilaista ongelmanratkaisua ja hyödyntää järjestelmältä eri kyvykkyyksiä. Ensimmäisessä tapauksessa malli tunnistaa valmiiksi määritellyn metriikan ja osaa käyttää sitä suoraan, kun taas jälkimmäisessä tapauksessa sen on pääteltävä tarvittava laskentatapa. Tämän vuoksi tapaukset erotellaan analyysissa toisistaan, vaikka niiden semanttinen tavoite on sama.

Tuotettujen SQL-kyselyiden oikeellisuus arvioitiin manuaalisesti vertaamalla niitä tutkijan tulkintaan siitä, millainen kysely vastaisi käyttäjän tiedontarvetta. Arvioinnin suoritti yksi arvioija, mikä heikentää luokittelun reliabiliteettia.

5 Kokeet

Ensimmäisessä kokeessa tarkastellaan Cortex Analystin konsistenssia, eli sitä, tuottaako järjestelmä saman SQL-kyselyn, kun sama luonnollisen kielen kysymys esitetään useita kertoja.

Toisessa kokeessa tarkastellaan järjestelmän herkkyyttä sellaisille syötteen muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä. Näissä kokeissa muodostetaan useita eri sanamuotoja samasta tiedontarpeesta ja arvioidaan, tuottaako järjestelmä niistä olennaisesti saman SQL-kyselyn.

Konsistenssikoe Koska kielimallit ovat tilastollisia järjestelmiä, sama syöte ei välttämättä aina johda täsmälleen samaan tulosteeseen. Tietokantakyselyiden tapauksessa tämä on olennainen kysymys, koska käyttäjät odottavat järjestelmältä ennustettavaa ja toistettavaa toimintaa.

Konsistenssikokeessa sama luonnollisen kielen kysymys esitetään järjestelmälle 10 kertaa. Jokaisesta suorituksesta tallennetaan järjestelmän tuottama SQL-kysely, joita verrataan keskenään. Käytetyt kysymykset ovat seuraavat:

- A. How many total contacts did we handle in January 2026?
- B. What's our answer rate for calls in February 2026?
- C. What percentage of contacts that were answered within our service level target were closed by the customer versus closed by the agent?
- D. What was our peak contact hours by day of week? Show me the distribution of when contacts arrive throughout the day, broken down by hour and weekday.
- E. What's the success rate of our callbacks? Show me how many callbacks resulted in a conversation versus no answer, and what the average time between the original inbound contact and the callback was.

Määrittelyssä semanttisessa näkymässä on kuvattu metriikoihin SQL-kyselyt, jotka vastaavat suoraan kysymyksiin A ja B. Kysymykset C-E ovat huomattavasti monimutkaisempia, ja niihin ei ole ennalta validoituja SQL-kyselyitä semanttisessa näkymässä. Kysymys D vaatii, että kentästä `CONTACT_CONNECTED_DATETIME` erotetaan kellonaika tunnin tarkkuudella, ja viikonpäivä pitää hakea kalenteritaulusta. Kysymys E on rakenteeltaan monitulkintainen. Erityisesti takaisinsoiton ja alkuperäisen sisääntulevan kontaktin välisen ajan keskiarvoa koskeva osa voidaan tulkita vaativan kahden eri rivin välistä vertailua, vaikka käytetyssä aineistossa vastaus voidaan muodostaa yhdeltä takaisinsoittoa kuvaavalta riviltä.

Sensitiivisyyskoe Kokeen tavoitteena on arvioida, tulkitseeko Cortex Analyst semanttisesti saman sisältöiset kysymykset johdonmukaisesti vai johtavatko kielelliset erot erilaisiin SQL-kyselyihin. Jos järjestelmä on hyvin herkkä muotoilun vaihtelulle, luonnollisen kielen käyttö rajapintana muuttuu käyttäjän kannalta epävarmaksi. Jos taas järjestelmä tuottaa olennaisesti saman kyselyn eri sanamuodoista huolimatta, tämä tukee sen käytettävyyttä käytännön työssä.

Koe toteutetaan muodostamalla kustakin konsistenssikokeessa käytetystä kysymyksestä 4 vaihtoehtoista versiota, joissa pyritään säilyttämään kysymyksen merkitys, mutta muokkaamaan sanamuotoa. Järjestelmän tuottamat SQL-kyselyt tallennetaan ja vertaillaan keskenään. Käytetyt kysymykset ovat seuraavat:

A. Variaatioita kysymyksestä A:

1. What was the total number of contacts we handled in January 2026?
2. How many contacts did we handle overall in January 2026?
3. What is the total contact volume for January 2026?
4. How many total customer contacts were handled during January 2026?

B. Variaatioita kysymyksestä B:

1. What was our call answer rate in February 2026?
2. What percentage of incoming calls were answered in February 2026?
3. What percentage of calls were answered in February 2026?
4. How did our call answer rate perform during February 2026?

C. Variaatioita kysymyksestä C:

1. Of the contacts answered within our service-level target, what share were closed by the customer compared to those closed by the agent?
2. For interactions resolved within the service level target, what percentage were customer-closed versus agent-closed?
3. Within our service level target, how is the closure split: what percent of contacts were closed by customers versus agents?
4. Among contacts answered within SLA, what proportion ended with customer closure versus agent closure?

D. Variaatioita kysymyksestä D:

1. When do contacts most frequently arrive during the week? Break this down by weekday and hour.
2. Show the hourly contact-arrival distribution for every day of the week, highlighting which hours were highest on each weekday.
3. What times during the day (hour-by-hour) drive the most contacts, split by weekday. Please summarize peak hours and the overall distribution.
4. Across the week, when do contacts arrive most often? Provide an hour-by-hour breakdown for each weekday and identify the peak contact hours.

E. Variaatioita kysymyksestä E:

1. What percentage of our callback attempts lead to an actual conversation? Please break down callbacks that reached a conversation vs. those with no answer, and calculate the average delay from the original inbound to the callback.

2. How effective are our callbacks? Show the success rate by counting callbacks that resulted in a conversation versus no answer, and report the average time-to-callback from the initial inbound contact.
3. For our callback campaign, what's the outcomes split and average response speed? Provide counts of conversations vs. no-answer callbacks, the resulting success rate, and the average elapsed time between inbound contact and callback.
4. Can you analyze callback performance for me: success rate (conversation vs no answer) with the number of callbacks in each outcome category, plus the average time lag from the inbound contact to when the callback occurred?

Jokainen kysymys molemmissa kokeissa esitetään järjestelmälle 10 kertaa ja jokainen yksittäinen suoritus tehdään tyhjällä sessiolla. Testit suoritettiin 23.03.2026-28.03.2026 välisenä aikana käyttäen samaa semanttista mallia.

Kymmenen toistoa valittiin käytännöllisenä kompromissina, joka mahdollistaa vaihtelun havaitsemisen ilman kohtuuttoman suurta manuaalisen arvioinnin työmäärää.

6 Tulokset

Merkitään konsistenssikokeen kysymykset A-E ja herkkyysskokeen variaatiot A1-A4, B1-B4, C1-C4, D1-D4 ja E1-E4 listojen 5 ja 5 mukaisesti. Kokeiden tulokset on esitetty taulukoissa 2 ja 3. Taulukot kuvaavat kunkin kysymyksen osalta, kuinka monta kertaa järjestelmän tuottama SQL-kysely oli korrekti, melkein korrekti, virheellinen tai johti tilanteeseen, jossa SQL-kyselyä ei tuotettu lainkaan.

Konsistenssikokeessa vain kysymys C tuotti sekä korrekteja että virheellisiä SQL-kyselyitä. Kysymys B tuotti kaksi uniikkia SQL-kyselyä, joista toinen käytti semanttisessa näkymässä määriteltä metriikkaa `answer_percentage` suoraan ja toinen laski vastaavan prosenttiluvun erillisellä laskukaavalla.

Kysymykset A, D ja E tuottivat konsistenssikokeessa vain yhden uniikin lopputuloksen kymmenessä suorituksessa. Vaikka kysymys E tuotti jokaisella suorituskerralla virheellisen vastauksen siinä mielessä, että SQL-kyselyä ei muodostettu, konsistenssin näkökulmasta järjestelmä oli johdonmukainen, koska tuloste oli joka kerta sama. Kysymys E osoittaa, että konsistenssi ei yksin riitä kuvaamaan järjestelmän luotettavuutta.

Tulokset viittaavat siihen, että tässä tutkimusasetelmassa havaitut ongelmat olivat usein toistettavia, mikä on hyödyllinen havainto sekä käyttäjän että semanttisen näkymän kehittäjän näkökulmasta. Kysymys C kuitenkin osoittaa, että järjestelmä pystyi tuottamaan samassa asetelmassa sekä korrekteja että virheellisiä vastauksia, joten toistettavuus ei ollut täydellistä.

Alla on esimerkki kysymykseen E tuotetusta ei kyselyä -vastauksesta:

I'm sorry, but this question cannot be fully answered with the available data. While we can determine how many callbacks were answered versus not answered (using the `c_answered` field filtered for callback media type), the semantic model does not contain information about the original inbound contact that triggered a callback. There is no field linking a callback conversation back to its originating inbound contact, so we cannot calculate the time between the original inbound contact and the callback attempt.

Kysymykseen B järjestelmä tuotti kaksi uniikkia SQL-kyselyä. Näistä toinen käytti semanttisessa näkymässä ennalta määriteltä metriikkaa `answer_percentage` ja toinen muodosti vastaavan laskukaavan eksplisiittisesti SQL:ssä. Kysymykseen C järjestelmä tuotti kaksi uniikkia SQL-kyselyä, joista toinen oli korrekti ja toinen virheellinen. Korrekti kysely tuotettiin seitsemän kertaa ja virheellinen kolme kertaa. Virheellisessä kyselyssä järjestelmä sekoitti asiakkaan ja agentin roolit keskenään.

Konsistenssikokeen 50 suorituksesta 37 tuotti korrektin SQL-kyselyn, 10 johti tilanteeseen, jossa järjestelmä ei tuottanut lainkaan SQL-kyselyä, ja 3 tuotti virheellisen SQL-kyselyn. Tulokset osoittavat, että Cortex Analyst pystyy usein tuottamaan johdonmukaisia SQL-kyselyitä, mutta sen luotettavuus ei ollut kaikissa kysymystyypeissä riittävällä tasolla.

Herkkyystesteissä suurin variaatio havaittiin kysymyksen C eri muotoiluissa, joissa järjestelmä tuotti yhteensä kuusi uniikkia SQL-kyselyä. Kun Cortex Analystille syötettiin kysymyksen C eri variaatioita, järjestelmä tuotti väärän vastauksen 14

kertaa 50 suorituksesta. Virheellisissä kyselyissä toistuvien ongelmien oli se, että järjestelmä ei rajannut hakua sarakkeen `direction` mukaisesti, vaan käytti saraketta `direction_original` tai jätti suuntasuodatuksen kokonaan pois.

Kysymyksessä C ja sen variaatioissa puhutaan palvelutasosta, jolloin impliittisenä taustaoletuksena on, että tarkastelu kohdistuu vain saapuviin kontakteihin. Havainto viittaa siihen, että Cortex Analyst ei tässä tutkimusasetelmassa aina huomionnut tällaisia implisiittisiä oletuksia johdonmukaisesti.

Kun kysymys E ja sen variaatiot annettiin järjestelmälle syötteenä, järjestelmä tuotti 40 kertaa 50 suorituksesta tekstivastauksen, jossa se ilmoitti, että kysymykseen ei voida vastata. Variaatio E2 tuotti yhdeksän kertaa korrektein SQL-kyselyn ja kerran kyselyn, joka oli muuten oikea mutta käytti `distinct(identifier)`-ehdon sijaan ehtoa, jossa koko rivi vaadittiin uniikiksi. Tämä on huomattavasti heikompi ehto ja johti duplikaattivirheisiin.

Kysymyksen E variaatio E2 ei implikoi yhtä vahvasti, että kysymykseen vastaaminen vaatisi kahden rivin välistä vertailua. Tämä voi osaltaan selittää, miksi juuri tämä variaatio tuotti enemmän oikeita vastauksia kuin muut.

Kysymys E oli Cortex Analystille vaikein, mikä painottaa virhetyyppien jakaumaa ei-kyselyä -luokan suuntaan.

Taulukosta 4 nähdään, että muista virhetyypeistä yleisimpiä olivat suodatusvirheet ja duplikaattien hallintaan liittyvät virheet. Kaikki suodatusvirheet liittyivät sarakkeisiin `direction` ja `direction_original`, mikä viittaa siihen, että järjestelmä voisi olla robustimpi, jos nämä kaksi saraketta erotettaisiin toisistaan semanttisesti selvemmin. Esimerkiksi `direction_original`-sarakkeen korvaaminen `is_callback`-sarakkeella saattaisi helpottaa mallia erottamaan sarakkeiden merkitykset toisistaan. Tätä ei kuitenkaan testattu tässä tutkimuksessa.

On kuitenkin huomioitava, että juuri nämä sarakkeet nousivat esiin tässä tutkimusasetelmassa. Toisenlaisessa tietokannassa tai toisessa sovellusympäristössä ongelmalliset kentät voisivat olla erilaisia. Havainto korostaa semanttisen näkymän merkitystä järjestelmän luotettavuuden kannalta. Semanttisen näkymän huolellinen suunnittelu ja iteratiivinen kehittäminen voivat pitkällä aikavälillä parantaa järjestelmän toimintaa merkittävästi.

Duplikaattien hallintavirheillä tarkoitetaan tässä virheitä, joissa `distinct`-avainsanaa on käytetty tarpeettomasti tai väärin. Tutkimusasetelmassa vain kysymystä A vastaavissa SQL-kyselyissä `distinct`-avainsanaa ei tarvita. Muiden kysymysten kohdalla duplikaattien hallinta tulisi yleensä kohdistaa `identifier`-sarakkeeseen. Duplikaattien hallintavirheitä esiintyi tutkimuksessa yhteensä 18 kertaa 250 suoritusten aikana.

Kysymyksessä A duplikaattien hallintaa ei odoteta lainkaan, mutta 10 suorituksessa järjestelmä tuotti SQL-kyselyn, joka sisälsi lausekkeen `distinct(identifier)`. Muut havaitut duplikaattien hallintavirheet liittyivät siihen, että `distinct`-avainsanaa sovellettiin n-tupleen, joka sisälsi `identifier`-sarakkeen lisäksi myös muita sarakkeita. Tällöin kysely vaatii näiden sarakkeiden yhdistelmän olevan uniikki, mikä on heikompi ehto kuin se, että `identifier`-sarake olisi jo yksin uniikki.

Taululiitos- tai sarakkeevalintavirheitä ei havaittu. Tämä viittaa siihen, että järjestelmä tunnisti kysymyksistä yleensä oikein, mitä attribuutteja käyttäjä halusi hakea, vaikka se ei aina onnistunut tuottamaan oikeaa SQL-kyselyä niiden perusteella.

Taulukko 2: Konsistenssikokeen tulokset.

Kysymys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Erilaisten SQL- kyselyiden määrä
A	10	0	0	0	1
B	10	0	0	0	2
C	7	0	3	0	2
D	10	0	0	0	1
E	0	0	0	10	1

Taulukko 3: Herkkyystestien 1–5 tulokset. Sarakkeet ilmoittavat oikeiden, pienen virheen sisältävien, väärin ja ei kyselyä -vastausten lukumäärät sekä yhteenlaskettu uniikkien SQL-kyselyiden määrä. (*Yht.*).

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
A	10	0	0	0	1
A1	10	0	0	0	1
A2	10	0	0	0	1
A3	10	0	0	0	1
A4	0	10	0	0	2

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
B	10	0	0	0	2
B1	10	0	0	0	2
B2	10	0	0	0	2
B3	10	0	0	0	2
B4	10	0	0	0	2

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
C	7	0	3	0	2
C1	8	2	0	0	3
C2	10	0	0	0	3
C3	0	0	10	0	5
C4	8	1	1	0	6

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
D	10	0	0	0	1
D1	10	0	0	0	1
D2	10	0	0	0	1
D3	10	0	0	0	2
D4	7	0	3	0	3

Kysmys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Yht.
E	0	0	0	10	1
E1	0	0	0	10	1
E2	9	1	0	0	3
E3	0	0	0	10	3
E4	0	0	0	10	3

Taulukko 4: Esiintyneiden virhetyyppien määrät kaikissa kokeissa. Rivit kuvaavat, kuinka monta kertaa kukin virhetyyppi esiintyi kussakin kysymyksessä tai sen variaatioissa. Jos yhdessä SQL-kyselyssä esiintyy useampi virhetyyppi, kysely lasketaan jokaiseen virhetyyppiin erikseen.

Kysymys	A	B	C	D	E	Yhteensä
Aggregaatio	0	0	2	0	0	2
Duplikaattien hallinta	10	0	7	0	1	18
Ei kyselyä	0	0	0	0	40	40
Sarakevalinta	0	0	0	0	0	0
Suodatus	0	0	11	3	0	14
Taululiitos	0	0	0	0	0	0

7 Rajoitukset

Tutkimukseen liittyy useita rajoituksia, jotka on huomioitava tuloksia tulkittaessa.

Ensinnäkin tutkimus kohdistuu yhteen järjestelmään, Snowflaken Cortex Analyysiin, jonka taustalla toimii Anthropicin Claude Sonnet 4 -malli. Tulokset kuvaavat siten yhtä tiettyä toteutusta eivätkä ole suoraan yleistettävissä muihin kielimalleihin, SQL-avustajiin tai luonnollisen kielen analytiikkatyökaluihin.

Toiseksi tutkimus perustuu yhteen sovellusalueeseen eli puhelinkeskusdataan. Tämän tietokannan rakenne, sanasto ja analyysitarpeet voivat poiketa merkittävästi muiden toimialojen vastaavista ympäristöistä. On mahdollista, että järjestelmä suoriutuisi eri tavoin esimerkiksi talousdatan, lokidatan tai asiakastietojen yhteydessä.

Kolmanneksi tutkimuksessa tarkastellaan mallin tuottamia SQL-kyselyitä eikä kyselyiden palauttamia varsinaisia vastauksia. Tämä rajaus on perusteltu aineiston luottamuksellisuuden vuoksi, mutta samalla se tarkoittaa, että tutkimus ei mittaa suoraan liiketoiminnallisen lopputuloksen oikeellisuutta. Kysely voi olla rakenteeltaan puutteellinen tai vaihtoehtoisesti syntaktisesti erilainen mutta käytännössä toimiva, eikä tätä eroa voida aina arvioida täysin ilman varsinaista tulosaineistoa.

Neljänneksi semanttisen näkymän laatu vaikuttaa suoraan siihen, miten hyvin Cortex Analyst kykenee muodostamaan kyselyitä. Jos näkymässä on epäselvyyksiä, puutteita tai monitulkintaisia kuvauksia, osa havaituista ongelmista voi johtua itse määrittelystä eikä pelkästään kielimallin rajoituksista. Tämän vuoksi tutkimus arvioi samalla myös sitä, kuinka hyvin kyseinen semanttinen näkymä tukee luonnollisen kielen ja tietokantarakenteen välistä tulkintaa.

Viidenneksi kielimallien toimintaa luonnehtii tietty epädeterministisyys, minkä vuoksi tulokset voivat vaihdella eri suorituskerroilla tai järjestelmäversioiden välillä. Lisäksi kaupalliset kielimallipohjaiset järjestelmät voivat muuttua ajan myötä ilman, että kaikki muutokset ovat käyttäjälle näkyviä. Tämän vuoksi tutkimuksen havainnot kuvaavat järjestelmän toimintaa tietyssä ajankohtana ja tietyssä konfiguraatiossa eivätkä välttämättä pysyvää ominaisuutta.

Kuudenneksi tuotettujen SQL-kyselyiden oikeellisuuden arvioi yksi arvioija. Tämä heikentää luokittelun reliabiliteettia ja lisää mahdollisuutta siihen, että osa luokitteluista sisältää arvioijan tulkintaan liittyvää subjektiivisuutta.

8 Johtopäätökset

Tämän tutkielman tavoitteena oli arvioida, kuinka hyvin Snowflaken Cortex Analyst soveltuu luonnollisen kielen tietokantakyselyihin puhelinkeskusdataa sisältävässä tietokantaympäristössä. Tarkastelun kohteena olivat erityisesti järjestelmän kyky tuottaa oikeellisia SQL-kyselyitä, sen konsistenssi tilanteissa, joissa sama kysymys esitetään useita kertoja, sekä sen herkkyyks sellaisille syötteen muotoilun muutoksille, joiden ei pitäisi muuttaa kysymyksen merkitystä.

Tulokset osoittavat, että Cortex Analyst kykenee monissa tapauksissa tuottamaan oikeellisia ja keskenään johdonmukaisia SQL-kyselyitä. Erityisesti yksinkertaisemmat kysymykset sekä sellaiset kysymykset, joihin oli määritelty valmiita metriikoita semanttisessa näkymässä, tuottivat hyvin vakaita tuloksia. Tämä viittaa siihen, että kielimallipohjainen luonnollisen kielen rajapinta voi toimia käyttökelpoisena välineenä tietokannan hyödyntämisessä silloin, kun kysymykset ovat rakenteeltaan selkeitä ja semanttinen näkymä tukee niitä riittävän hyvin.

Tutkimus osoitti kuitenkin myös merkittäviä rajoituksia. Järjestelmän suorituskyky heikkeni erityisesti silloin, kun kysymykseen sisältyi implisiittisiä taustaoletuksia tai kun oikean SQL-kyselyn muodostaminen edellytti tarkkaa suodatusta, duplikaattien hallintaa tai monitulkintaisen sanamuodon tulkintaa. Lisäksi havaittiin, että semanttisesti lähes saman sisältöiset kysymykset saattoivat johtaa erilaisiin SQL-kyselyihin ja joissakin tapauksissa myös erilaisiin virheisiin. Tämä heikentää luonnollisen kielen rajapinnan ennustettavuutta käyttäjän näkökulmasta.

Keskeinen havainto on, että järjestelmän luotettavuus ei riipu yksinomaan taustalla olevasta kielimallista, vaan myös siitä, miten hyvin semanttinen näkymä on suunniteltu. Tutkimuksessa havaitut virheet liittyivät usein tilanteisiin, joissa tietokannan kenttien merkitysten erottelu ei ollut mallin näkökulmasta riittävän selkeä. Tämän perusteella voidaan päätellä, että semanttinen näkymä ei ole pelkästään tekninen apurakenne, vaan keskeinen osa koko järjestelmän toimintaa. Sen huolellinen suunnittelu, yksiselitteinen nimeäminen ja iteratiivinen kehittäminen voivat parantaa luonnollisen kielen kyselyjärjestelmän luotettavuutta merkittävästi.

Tutkimuksen perusteella konsistenssia ei myöskään voida pitää yksinään riittävänä laadun mittarina. Järjestelmä saattoi tuottaa saman tuloksen toistuvasti myös silloin, kun tulos oli virheellinen tai kun SQL-kyselyä ei tuotettu lainkaan. Toisaalta joissakin tapauksissa samaan kysymykseen saatiin useita erilaisia SQL-kyselyitä, joista osa oli oikeellisia ja osa virheellisiä. Tämä osoittaa, että luonnollisen kielen tietokantarajapintojen arvioinnissa on tarkasteltava samanaikaisesti sekä oikeellisuutta että vaihtelua.

Kokonaisuutena tutkielma tukee näkemystä, että luonnollisen kielen käyttö tietokantojen rajapintana on lupaava mutta vielä osin epävarma lähestymistapa. Cortex Analyst voi toimia tehokkaana työkaluna erityisesti rajatuissa käyttötapauksissa, joissa tietokannan rakenne tunnetaan hyvin ja semanttinen näkymä on laadittu huolellisesti. Sen sijaan tilanteissa, joissa kysymykset sisältävät implisiittisiä oletuksia, monitulkintaisuutta tai useita päättelyvaiheita, järjestelmän tuottamiin kyselyihin on syytä suhtautua varauksella.

Tutkielman tulokset viittaavat siihen, että käytännön kehitystyössä huomio kannattaa kohdistaa paitsi mallin suorituskykyyn myös siihen, miten järjestelmälle tar-

jotta semanttinen konteksti on rakennettu. Tulevassa tutkimuksessa olisi hyödyllistä vertailla useita erilaisia semanttisia näkymiä, testata järjestelmää muissa sovellusympäristöissä sekä tarkastella tuotettujen SQL-kyselyiden lisäksi myös niiden palauttamien vastausten oikeellisuutta. Lisäksi useamman arvioijan käyttö voisi parantaa luokittelun reliabiliteettia ja vahvistaa tulosten luotettavuutta.

Tämän tutkimuksen perusteella voidaan päätellä, että kielimallipohjaiset luonnollisen kielen tietokantarajapinnat eivät vielä täysin korvaa asiantuntijan laatimia kyselyitä, mutta ne voivat jo nyt toimia hyödyllisenä tukena analytiikkatyössä. Niiden käytännön arvo riippuu kuitenkin ratkaisevasti siitä, kuinka hyvin järjestelmän toimintaympäristö, semanttinen malli ja käyttötapa on sovitettu yhteen.

Viitteet

- [1] Newton ja Leibniz
- [2] Riemann
- [3] Snowflake Inc. (n.d.). Cortex Analyst. Retrieved April 22, 2026, from <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst>
- [4] Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). Neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137–1155.
- [5] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [7] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft).
- [8] Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.
- [9] Mikolov, T., Chen, Kai, Corrado, G. & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*.